

Parallel Graph Coloring for Task-Scheduling Simulation

Jason Yuan (laiy), Rui Zhou (ruizhou)

1 Url

<https://jyuan2003.github.io/task-scheduler>

2 Summary

We implemented and optimized a parallel scheduler that maps tasks to different processes under both OpenMP and MPI on CPU. We implemented a graph generator which encodes the computation and communication of tasks into a graph and represented it in four graph representations. We analyzed the performance and scalability of our implementation for varying graphs, batch sizes, and graph representations.

3 Background

In parallel programming, the speedup of the code can often depend on the effectiveness of the task assignment policy. On the one hand, it is often desirable to ensure that each process gets about the same amount of computation, so that the process with the most tasks assigned to it does not become a bottleneck to the total speedup of the program; on the other hand, the computation of task i may produce a result that is required in the computation of task j , so assigning task i and j to different processes can induce overhead such as synchronization or communication. One challenge to implementing an effective task scheduler is that the requirement to balance workload and the requirement to minimize data dependency across processes can often go against each other. Another issue is that the workload and dependency of a task can be dynamically changing (a good example of which is the Barnes-Hut algorithm), so the scheduler will also need to dynamically adjust its task scheduling.

In this project, we will implement a parallel task scheduler as follows: given N tasks and P processes, we construct a graph G in which each task i is represented as a vertex v_i , and task i 's data dependency on task j is represented as an edge (v_i, v_j) . Given P parallel processes, process i is represented by the color P_i , and given a coloring χ of the graph, the cost of vertex v_i is defined by

$$\begin{aligned} c(v_i) &= \text{computation time of task } i + \sum_{(v_i, v_j) \text{ s.t. } \chi(v_i) \neq \chi(v_j)} \text{data dependency on task } j \\ &:= t(v_i) + \sum_{(v_i, v_j), \chi(v_i) \neq \chi(v_j)} t(v_i, v_j) \end{aligned}$$

That is, we sum up the dependency on the tasks that are assigned to a different process and the computation time of the task itself. For each P_i , the total time of process i is

$$t(P_i) = \sum_{\chi(v_j)=P_i} c(v_j)$$

and the total time of the program is $t(G) = \max_i t(P_i)$. We will approximate an optimal scheduling of tasks as follows:

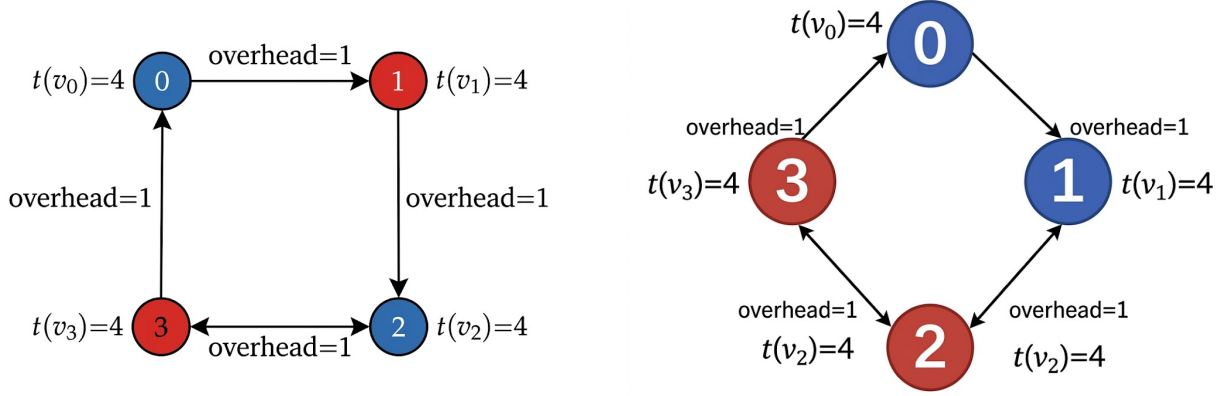
```

initially color G
partition G into NUM_THREADS parts
//each thread does the following in parallel
for (int i = 0; i < NUM_IT; i++){
    for v_j in the partition{
        for (int k = 0; k < P; k++){
            color v_j with P_k
            compute t(G) under this coloring
        }
        update the color of v_j to the one that gives lowest cost
    }
}

```

Note that when the color of v_j is updated, the only changes that this update makes to $t(G)$ are at v_j and its neighboring vertices, so we only need to go through the neighbors of v_j to compute the new value of $t(G)$.

Below is a toy example of the result after running one iteration of the algorithm:



To handle update queries, we simply modify the algorithm by changing the graph structure for each update and modify the cost computation accordingly.

This algorithm can greatly benefit from parallel programming when the number of tasks N is large, since computation and update of a vertex only involves its neighboring vertices without affecting the cost of other vertices, so there is a lot of room for parallelism.

3.1 Graph Representations

- Raw Graph: A data structure consisting of unprocessed nodes and edges.
- Sharded Graph: A raw graph is decomposed into a vector of sharded graphs. Each sharded graph contains a contiguous interval of nodes. For these nodes, we only store their incoming edges and sort them by the source vertex id.
- Compressed Graph: For each vertex, we sort its outgoing edges and normalize it with respect to the vertex. We group them based on the number of bytes.

- **Sharded Compressed Graph:** We decompose the graph into a vector of sharded graphs. We compress each sharded graph to get the sharded compressed graph.

Shared APIs

- `num_nodes()`: Returns the number of nodes in the graph.
- `num_edges()`: Returns the number of edges in the graph.
- `get_node_weight(x)`: Returns the weight (compute cost) of node `x`.
- `get_edge_weight(u, v)`: Returns the weight (communication cost) of edge `u -> v`.
- `get_node_in_neighbors(u)`: Returns a vector of nodes `v` such that `v -> u`.
- `get_node_out_neighbors(u)`: Returns a vector of nodes `v` such that `u -> v`.
- `local_to_global(x)`: Returns the global id corresponding to `x`.

4 Approach

4.1 Data Generation

We build raw graphs using `networkx` in Python with the following attributes:

- **topology:** We consider two topologies (1) Random graphs, where nodes and edges are randomly added and (2) Clustered graphs.
- **size:** The number of nodes in the graph.
- **density:** Defines the how dense the graph is. There are 3 modes of densities: sparse, medium, and dense. For random graphs, they correspond to n , $n \log_2 n$, and $10n \log_2 n$ edges, where n is the number of nodes. For clustered graphs, they are controlled by the probabilities of adding an edge within or between the clusters.

4.2 Graph Representations

We implement the representation conversion functions in C++:

- `raw_to_sharded()` converts a raw graph into a vector of sharded graphs.
- `raw_to_compressed()` converts a raw graph into a compressed graph.
- `sharded_to_compressedsharded()` converts a sharded graph into a sharded compressed graph.

Along with other utility functions we implemented, including load and save, we can convert a raw graph into the remaining three representations as described in 3.1.

4.3 MPI

Recall that we represent each node as a task. The weight of a node represents the computation time, and the weight of an edge represents the communication cost.

We explored two potential formulations.

We started with our first formulation. It models our task scheduler more closely to a real system, where we keep the CPU busy to mimic the node weight, and send dummy messages of varying sizes to mimic the edge weight. Each process only tries to update its own nodes, and in order to see if a different color can lead to better runtime span, it sends its nodes to the processes that own their neighbors and determines a

better color after receiving the results. However, we found it difficult to determine how long the CPU should stay busy, how large the messages should be, and how frequently the messages should be sent in order to match the provided weight from the graph. We tried different simulation implementations as well and found that it is generally more difficult to distinguish the effects of our communication strategy from those of the simulation implementation itself.

This shifts the focus to our next formulation, and our results are in section 5. This formulation is more similar to the wire routing task, where instead of simulating the computation and communication, we assume that the compute and communication cost are known. Each process keeps a local copy of color assignment and runtime. It iterates over its nodes and evaluates different colors by reading the required costs directly from the graph. Once it has processed a number of nodes and accumulated `batch_size` number of updates, it asynchronously sends the updates to other processes. It also checks if there are any pending messages, and if so, update its own copy of color assignment. It recomputes local scheduler after a period of time.

We explored two **asynchronous** communication strategies. We first tried to let a process broadcasts the color updates to other processes. However, we didn't observe satisfactory scaling behavior. When we profiled the implementation, we observed that the wall-clock compute time increased as the number of processes increased. We believe this is because the number of messages in this communication strategy scales with the number of processes.

Unlike the broadcast-based strategy, the ring-based strategy forwards updates through a logical process ring instead of sending each update directly to every process. We adapted the communication implementation from our assignment 4 implementation. Specifically, we store 3 buckets: `sendbufs`, `recv_updates`, `fw_updates`. `sendbufs` holds locally generated updates, `recv_updates` holds newly received updates from its left neighbor, and `fw_updates` holds received updates that are being forwarded to its right neighbor. In addition, we send messages only when we have accumulated `batch_size` number of updates, which can reduce the repeated small-message overhead.

We tested our MPI implementations on our local machine equipped with an AMD Ryzen 9 9950X3D processor, which has 16 physical cores and 32 software threads.

For the experiment, we didn't consider problem-constrained scaling, since in our task, more processes also means more compute. Instead, we considered time-constrained scaling, which, according to the lecture slides, is defined as

$$\text{Speedup} = \frac{\text{work done by } P \text{ processors}}{\text{work done by 1 processor}}.$$

We represent the workload by the number of nodes in the graph. For each trial, we use a 30-second time limit and run binary search to find the maximum number of nodes that can be solved within that limit.

4.4 OpenMP

In our OpenMP implementation, since the computation model is based on shared memory, it is more convenient for us to experiment different task-allocation policies and compare their performance.

Our first naïve attempt is to allocate a contiguous chunk nodes of roughly the same size to each processor and let each processor only completes its own portion of the task. Although this has rather acceptable performance on random graphs, the workload balancing becomes quite poor when the input data has some high-degree nodes or when the graph becomes clustered.

We tried to optimize our task assignment policy by measuring the total degree of a batch of nodes, and assign batches to the processors in a way such that each process gets roughly the same number of total degree. However, this approach reaches a dead end, as the initialization process is already $O(n)$ per node, thus $O(n^2)$ in total. When timing the run-time, initialization becomes the bottleneck to this approach, taking more than 10 seconds on moderate-sized graphs.

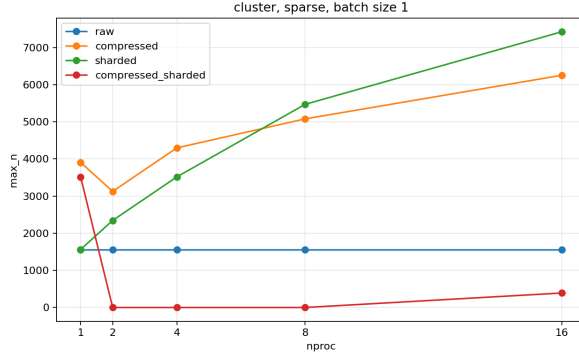
Similarly, we tried to improve the cache-miss rate on specific input graphs by first coloring the input graph for a few iterations (which should hopefully be sufficient to separate the nodes into different clusters), then re-index the nodes based on the cluster they are in. However, although this approach has relatively less overhead compared to the previous one, the benefit of reassignment is still outweighed by the extra computation.

We then experiment with dynamic job-stealing, in which we have a vector `bucket_done` indicating whether the i^{th} bucket has already been computed in the current iteration, and if not then that bucket can be dynamically moved to another processor. This indeed improves the performance significantly, since now processors that happen to get low total degree do not need to stay idle.

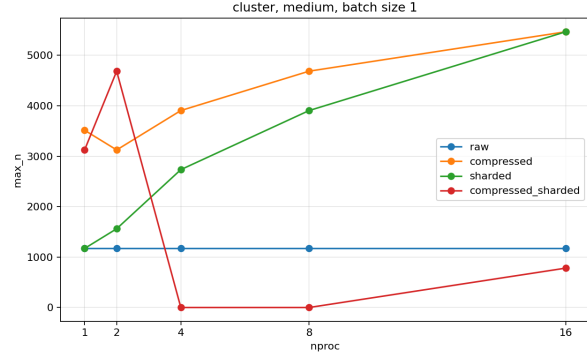
We finally consider improving our update policy. Initially, there is only one lock for the single scheduler, and after computing the color of a single node, the processor needs to grab the lock and update the scheduler, which can in many cases serialize the computation. To circumvent this, we instead keep a local knowledge of the updated graph-coloring and store it in a vector `local_color`, then update the nodes in the vector altogether. This significantly reduces the synchronization time.

5 Results

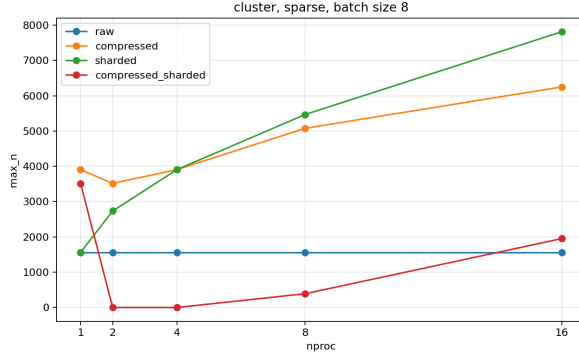
As described in section 4.3, we measured the maximum number of nodes that can be solved within a fixed period of time. Given a fixed graph topology, edge density, and batch size, we use binary search to generate graphs with different numbers of nodes. Each trial is run with a 30-second time limit. If a trial times out, we try a smaller graph. Otherwise, we try a larger graph. We evaluate our ring-based implementation by comparing the maximum solvable graph size for a fixed graph type and batch size (1, 8, 128) across different graph representations (raw, sharded, compressed, sharded + compressed) and numbers of processes (1, 2, 4, 8, and 16).



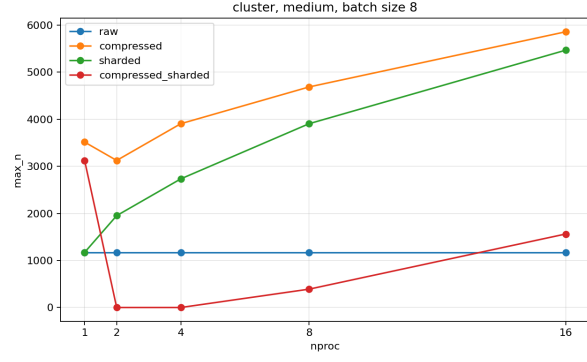
(a) Sparse, $b = 1$



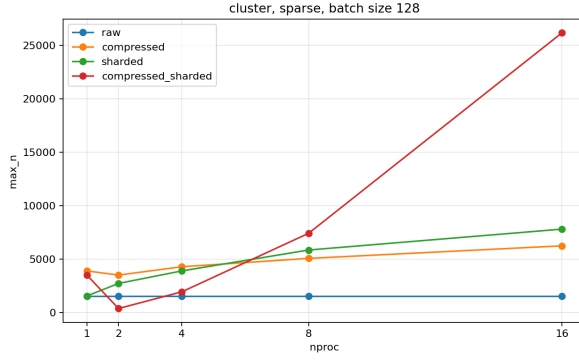
(b) Medium, $b = 1$



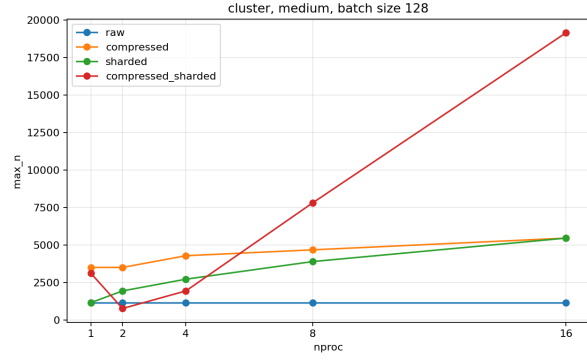
(c) Sparse, $b = 8$



(d) Medium, $b = 8$



(e) Sparse, $b = 128$



(f) Medium, $b = 128$

Figure 1: Cluster topology under sparse and medium edge densities.

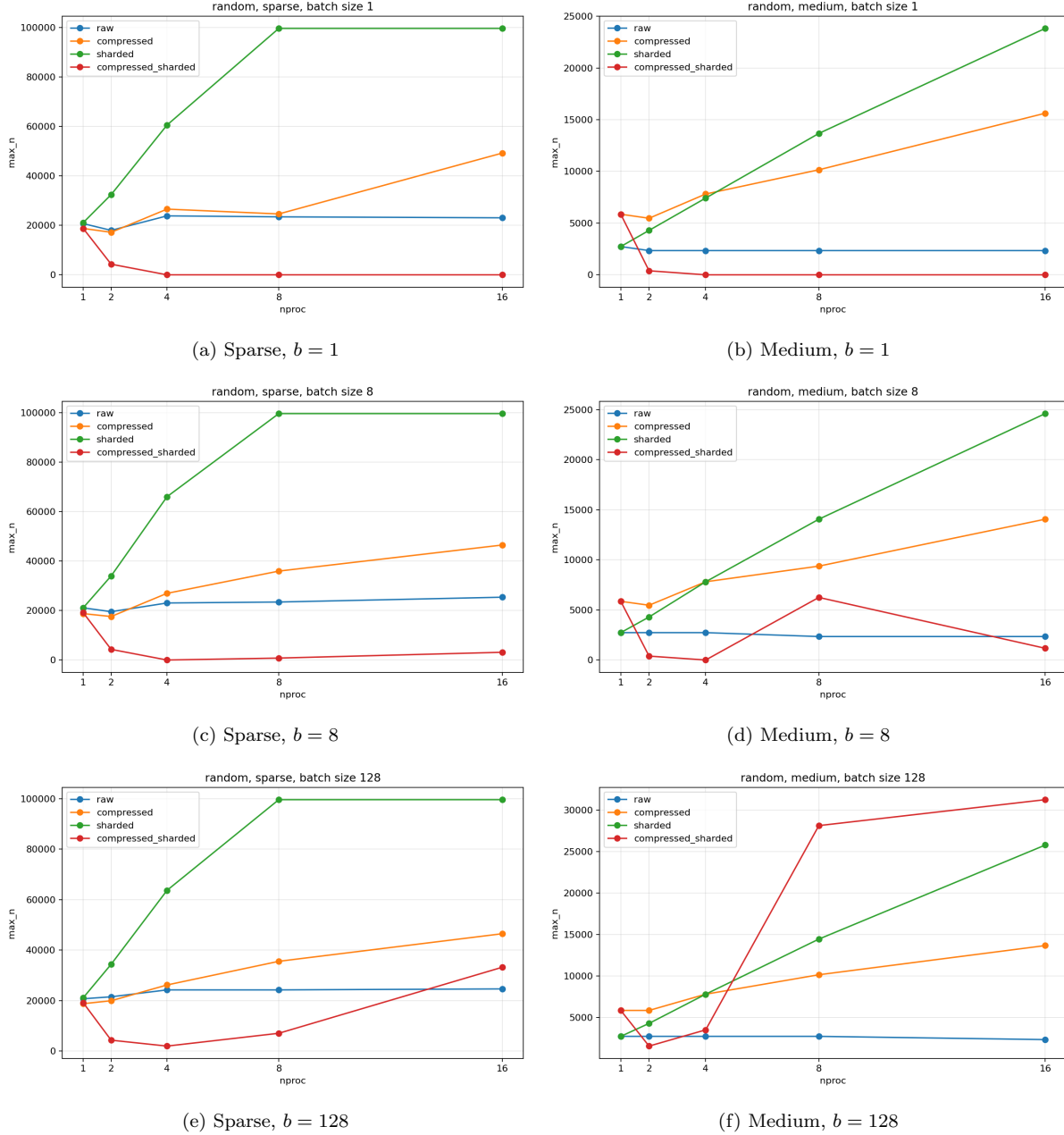


Figure 2: Random topology under sparse and medium edge densities.

As shown above, our implementation generally exhibits decent scalability. Although in a few settings, raw and compressed + sharded representations failed to show evident scaling-up behavior, and in some cases even degraded as the number of processes increased, we believe this is expected even under the time-constraint scaling setting. We believe that the total work can be represented in terms of graph size and the number of processes, so the work done per process per unit time is approximately same as the graph size. This makes the maximum feasible graph size bounded by a constant.

At $P = 1$, for graph of size ~ 10000 , the graph accessor and iterator take around the entire time (97%) for the raw and sharded representation. And as the graph size approaches ~ 100000 , then the graph accessor and iterator take around 100% time. Raw representation scales poorly because each accessor searches over the entire edge space. On the other hand, sharded representation scales better as it searches over a

smaller edge space. As P increases, communication and synchronization dominate. For instance, at graph of size ~ 10000 , the update of scheduler and message passing and receiving takes about 0.29s when the runtime is only about 0.30s

Our choice of machine is target sound, because our algorithm involves irregular graph traversal, and there is non-trivial amount of dependencies and condition checks, which can lead to warp divergence if we switch to GPU.

6 Work Distribution

We split the work relatively evenly (60%-40%).

- Graph generator and representations (Jason)
- MPI implementations (Jason)
- OpenMP implementations (Rui)